Project Title : Comparative Fake News Detection Using Statistical and Embedding-Based NLP Models

Selected Domain : Fake News Detection

Student Name : Vinaya Senthilkumar

Module : Natural Language Processing

1. Domain-Specific Area

This project focuses on **fake news detection** as a domain-specific Natural Language Processing task. Fake news can be understood as misleading, fabricated or unreliable news content that is presented in a format that may appear credible to readers. This is an important problem because digital news articles can spread rapidly through online platforms, making it difficult for readers, journalists and content moderators to manually verify every article. Automated text classification can therefore support the identification of unreliable news content by analysing linguistic patterns within article titles and body text.

Fake news detection is suitable for Natural Language Processing because news articles are primarily text-based and may contain linguistic features that distinguish fake articles from real ones. Recent research describes fake news as a significant digital information problem because false or misleading news can spread widely through modern online platforms and influence public understanding of events [1]. Machine learning and deep learning techniques are therefore commonly investigated as automated approaches for identifying fake news from textual and contextual features [1].

This task is challenging because fake news may be written in a style that imitates genuine news, meaning that simple keyword matching is unlikely to be sufficient for reliable classification. Recent comparative research has shown that fake news classification can be approached using traditional machine learning, convolutional neural networks, transformer-based models and large language models, with each method offering different strengths and limitations [2]. This supports the need to compare more than one modelling approach rather than relying on a single classifier.

However, fake news detection is not only a modelling problem; it is also an evaluation problem. A classifier may achieve high accuracy by learning dataset-specific shortcuts rather than genuine indicators of misinformation. This is especially important when fake and real articles come from different sources or collection methods. In such cases, the model may learn source-related patterns instead of learning whether the content itself is reliable. The selected dataset is related to the ISOT fake news dataset structure, where truthful articles are reported as being collected from Reuters, while fake articles are collected from unreliable

websites flagged by fact-checking sources [3]. This creates a possible source artefact that must be considered when interpreting model performance.

This domain is appropriate because it allows a clear comparison between statistical and embedding-based text classification models. A traditional statistical model can represent news articles using word-frequency features such as CountVectorizer or TF-IDF, while an embedding-based deep learning model can learn dense word representations and phrase-level patterns. Therefore, fake news detection provides a meaningful classification problem for evaluating whether simpler statistical NLP methods or neural embedding-based models are more effective for binary news classification. At the same time, the project allows critical discussion of whether high model scores reflect genuine fake news detection ability or dataset-specific patterns.

2. Objectives of the Project

The main objective of this project is to develop and evaluate a fake news detection system using Natural Language Processing text classification methods. The project compares a traditional statistical machine learning approach with an embedding-based deep learning approach in order to examine their effectiveness for classifying news articles as fake or real. This objective directly supports the coursework requirement to compare statistical and embedding-based models for a domain-specific text classification task.

The first objective is to prepare the selected news dataset for machine learning. This involves combining the article title and body text, creating binary class labels, checking class distribution, removing missing or duplicate values, and applying text-cleaning techniques such as lowercasing, punctuation removal and whitespace normalisation. Numbers are retained because they may contain meaningful information in news articles, such as dates, financial values or event counts. These preprocessing decisions are important because fake news detection depends heavily on textual patterns, and noisy or inconsistent text may reduce model performance [1].

The second objective is to implement a traditional statistical model for fake news detection. This project uses a baseline model based on CountVectorizer with Multinomial Naive Bayes, followed by a stronger statistical model using TF-IDF with Logistic Regression. CountVectorizer represents text using word-count features, while TF-IDF represents text using word-importance scores. These models are useful because they are efficient, interpretable and commonly used for text classification tasks.

The third objective is to implement an embedding-based deep learning model using an Embedding layer and a 1D Convolutional Neural Network. For this model, tokenisation and sequence padding are applied after the train-test split so that article text can be converted into fixed-length numerical sequences. This differs from the statistical models, which use sparse word-frequency or TF-IDF feature matrices. The CNN model is included because

recent comparative studies of fake news classification show that convolutional neural networks can learn useful phrase-level patterns from text sequences [2].

The fourth objective is to compare the models using appropriate evaluation metrics, including accuracy, precision, recall, F1-score and confusion matrix analysis. Accuracy will show the overall proportion of correct predictions, while precision, recall and F1-score will provide a more detailed view of class-level performance. This is important because fake news detection should consider both false positives and false negatives rather than relying only on overall accuracy [4]. The comparison will help determine whether a statistical model or an embedding-based deep learning model performs better on the selected dataset.

The final objective is to examine whether high model performance may be influenced by dataset-specific artefacts. This is important because the selected dataset is related to the ISOT fake news dataset structure, where truthful articles are reported as being collected from Reuters while fake articles are collected from unreliable websites [3]. Therefore, an additional Reuters dateline ablation test will be used to check whether the models rely partly on source-related patterns rather than only genuine fake-news language patterns.

3. Dataset Description

The dataset selected is the **Fake and Real News Dataset**, which is publicly available on Kaggle [5]. This dataset is suitable because it provides a clear supervised binary classification task, where each news article can be classified as either fake or real. It is directly relevant to the project aim because it contains full article text rather than only short headlines, allowing both statistical and embedding-based NLP models to learn from meaningful textual content.

The dataset is given as two csv files: **Fake.csv** and **True.csv**. **Fake.csv** contains **23481 fake news articles** and **True.csv** contains **21417 real news articles**, giving a combined total of **44898 articles** before duplicate removal. After checking and removing **209 duplicate rows**, the final dataset used for modelling contains **44689 articles**. Each file contains four columns: **title**, **text**, **subject** and **date**. The **title** column stores the article headline, **text** column stores article body, **subject** column stores news category and **date** column stores publication date.

For this project, the **title** and **text** columns are combined into a single input feature called **content**. This is useful because the headline may contain attention-grabbing or signal styles, while the article body provides longer linguistic patterns for classification. The target label is created from the original file source: articles from **Fake.csv** are labelled as **0**, while articles from **True.csv** are labelled as **1**. This creates a binary supervised learning dataset suitable for training and evaluating fake news classifiers.

The dataset also supports the comparison of different text representation methods. For the statistical models, the article text is converted into sparse numerical features using CountVectorizer and TF-IDF. For the embedding-based model, the cleaned text is tokenised into word sequences and converted into dense vector representations using an Embedding

layer. This directly supports the coursework requirement to compare statistical and embedding-based text classification approaches.

Although the dataset is useful, it has important limitations. The labels are derived from the dataset files rather than being manually verified during this project. More importantly, the dataset is related to the ISOT fake news dataset structure, where truthful articles are reported as being collected from Reuters while fake articles are collected from unreliable websites [3]. This may create a source artefact, where models learn source-specific terms such as Reuters datelines rather than genuine misinformation patterns. Therefore, the dataset is suitable for developing a text-based fake news classifier, but the results must be interpreted cautiously and supported by an additional Reuters dateline ablation test.

4. Evaluation Methodology

The fake news models will be evaluated using supervised binary classification. After preprocessing, the dataset will be split into training and testing sets. The training set fits the models, while the test set evaluates performance on unseen articles and generalisation.

The main evaluation metrics used in this project are accuracy, precision, recall, F1-score and confusion matrix analysis. These metrics are selected because accuracy alone does not show whether the model performs equally well on both fake and real news classes. Recent machine learning evaluation research highlights the importance of selecting appropriate metrics for supervised classification tasks rather than relying on a single performance measure [4]. The formulas used for these metrics are based on true positives (TP), true negatives (TN), false positives (FP) and false negatives (FN) [4], [6].

Accuracy measures the overall proportion of correctly classified articles. It is calculated as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy is useful for understanding general performance, but it may hide class-level errors. For example, a high accuracy score does not necessarily show whether the model is better at detecting fake news or real news.

Precision measures how many articles predicted as a class are actually correct. It is calculated as:

$$Precision = \frac{TP}{TP + FP}$$

Precision is important because a fake news detection system should avoid incorrectly labelling real news as fake. A low precision score would mean that the model produces too many false positives.

Recall measures how many actual articles from a class are successfully detected by the model. It is calculated as:

$$Recall = \frac{TP}{TP + FN}$$

Recall is important because failing to detect fake news would reduce the usefulness of the classifier. A low recall score would mean that the model misses too many fake news articles.

The F1-score provides a balanced measure between precision and recall. It is calculated as:

$$F1\text{-}score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

The F1-score is useful when both false positives and false negatives matter. In fake news detection, this is important because classifying fake news as real may allow unreliable content to pass undetected, while classifying real news as fake may reduce trust in legitimate reporting.

A confusion matrix will also be used to examine the number of true positives, true negatives, false positives and false negatives produced by each model. This will allow the evaluation to identify whether a model is more likely to misclassify fake news as real or real news as fake. Such analysis is important because fake news detection is not only about achieving a high overall score, but also about understanding the type of errors made by the classifier.

The models will be compared using the same train-test split and the same evaluation metrics to ensure a fair comparison. The baseline model, statistical model and embedding-based model will be evaluated side by side using a results table and visualisations such as confusion matrices and bar charts. In addition, an ablation test will be used later in the project to examine whether Reuters dateline source artefacts influence model performance. This methodology will make it possible to assess both predictive performance and the reliability of the model comparison.

5. Data Preprocessing

5.1 Importing Libraries

In [4]:
```
# import Libraries

# handling data
import pandas as pd
import numpy as np

# text cleaning
import re
import string
```

```
# charts and visualisations
import matplotlib.pyplot as plt

# splitting data and building ML models
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression

# model evaluation
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatri

# deep learning model
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Conv1D, GlobalMaxPooling1D, Dense, D

# To make results reproducible( randome state use )
RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
tf.random.set_seed(RANDOM_STATE)
```

5.2 Loading Dataset

The first stage is to load the selected fake news dataset. The dataset is stored in two separate csv files which is Fake.csv that contains fake news articles and True.csv that contains real news articles. The files are loaded using the pandas library because it provides a convenient way to read, inspect and manipulate tabular data before applying machine learning methods.

It is important to load the files separately because the original dataset stores the two classes in different files rather than in one combined labelled dataset. Therefore the fake and real news articles must first be imported as separate DataFrames before class labels can be added and the datasets can be combined.

In [5]:
```
# load the fake and real news datasets

fake_df = pd.read_csv("Fake.csv")
real_df = pd.read_csv("True.csv")

# print the shape of the fake and real news dataset
# to know each dataset row and column
print("Fake news dataset shape:", fake_df.shape)
print("Real news dataset shape:", real_df.shape)
```

```
Fake news dataset shape: (23481, 4)
Real news dataset shape: (21417, 4)
```

The output shows the number of rows and columns in each dataset. The fake news dataset contains **23481 rows and 4 columns**. The real news dataset contains **21417 rows and 4 columns**. This shows that both the csv files have been loaded successfully and that the dataset contains a large number of labelled news articles for model training and evaluation.

The four columns represent the available features in the dataset. At this stage, the datasets are still separate, so the next step is to inspect the column names and then create a binary class label for each file. This is needed because supervised machine learning models need both input features and target labels.

5.3 Inspecting Dataset Columns

After loading the dataset, the next step is to inspect the column names in both csv files. This is important because the project needs to identify which columns should be used as model input features and which columns are not required for classification. In this dataset, the most important fields should be title and text, as these contain the headline and article body.

In [6]:
```
# check the columns in the fake and real datasets

# print columns in the fake dataset
print("Fake dataset columns:")
print(fake_df.columns)

# print columns in the real dataset
print("\nReal dataset columns:")
print(real_df.columns)
```

```
Fake dataset columns:
Index(['title', 'text', 'subject', 'date'], dtype='object')

Real dataset columns:
Index(['title', 'text', 'subject', 'date'], dtype='object')
```

The column inspection confirms that both datasets contain the same four columns: **title**, **text**, **subject** and **date**. This consistency is important because it means the fake and real datasets can be combined without structural issues.

For this project, the **title** and **text** columns will be used as the main textual input because they contain the language patterns needed for fake news classification. The **subject** and **date** columns will not be used as model input in the main classification task, because the aim is to evaluate text-based classification using article content rather than metadata.

5.4 Adding Class Labels

The original dataset stores fake and real news articles in two separate csv files. Before the datasets can be combined, a class label must be added to each DataFrame. This creates the target variable required for supervised machine learning.

For this, fake news articles are labelled as **0** while real news articles are labelled as **1**. This binary label structure allows the models to learn the difference between fake and real news during training.

In [7]:
```
# add the class labels to both datasets
# 0 represents fake news
# 1 represents real news

fake_df["label"] = 0
real_df["label"] = 1

# check if labels are added
# print the count for the labels in each dataset
print("Fake dataset label values:")
print(fake_df["label"].value_counts())

print("\nReal dataset label values:")
print(real_df["label"].value_counts())
```

```
Fake dataset label values:
label
0    23481
Name: count, dtype: int64

Real dataset label values:
label
1    21417
Name: count, dtype: int64
```

The output shows that the class labels have been added properly. All articles from the fake news dataset have been assigned the label **0** and all articles from the real news dataset have been assigned the label **1**.

This step is necessary because the machine learning models require a target variable to learn from. Without the **label** column the models would only get article text but would not know if each article belongs to the fake or real news class.

5.5 Combining the Fake and Real Datasets

After adding class labels, the fake and real news datasets need to be combined into one DataFrame. This is necessary because the machine learning models require one full complete dataset containing both the input text and the target labels.

The combined dataset will contain articles from both the classes. After combining, the rows are shuffled so that fake and real articles are mixed randomly. Shuffling is important because the original files are grouped by class, and training a model on an ordered class groups could result in bias during data splitting.

In [8]:
```
# combine the fake and real news datasets
```

```
news_df = pd.concat([fake_df, real_df], axis=0)

# shuffle news_df ( combined dataset) using random state 42
news_df = news_df.sample(frac=1, random_state=RANDOM_STATE).reset_index(drop=True)

# check shape of the combined dataset
print("Combined dataset shape:", news_df.shape)

# show first 5 rows
news_df.head()
```

```
Combined dataset shape: (44898, 5)
```

Out[8]:

	title	text	subject	date	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	21st Century Wire says Ben Stein, reputable pr...	US_News	February 13, 2017	0
1	Trump drops Steve Bannon from National Securit...	WASHINGTON (Reuters) - U.S. President Donald T...	politicsNews	April 5, 2017	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	(Reuters) - Puerto Rico Governor Ricardo Rosse...	politicsNews	September 27, 2017	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	On Monday, Donald Trump once again embarrassed...	News	May 22, 2017	0
4	Donald Trump heads for Scotland to reopen a go...	GLASGOW, Scotland (Reuters) - Most U.S. presid...	politicsNews	June 24, 2016	1

The combined dataset contains **44898 rows and 5 columns**. The additional column is the **label** column, which was created to identify if each article is fake or real. This shows that the fake and real news datasets have been merged into a single supervised learning dataset.

The dataset was also shuffled after merging. This ensures that fake and real articles are randomly distributed throughout the DataFrame rather than remaining grouped by class. This is important before splitting the data into training and testing sets because it helps create a fair distribution of classes across both sets.

5.6 Checking Class Distribution

Before training the models, it is important to check the distribution of the target classes. Class distribution shows how many fake and real news articles are present in the combined dataset.

This step is important because a highly imbalanced dataset can affect model performance. If one class appears much more often than the other the model may learn to favour the majority class. Checking the class distribution helps determine whether the dataset is suitable for fair binary classification.

In [9]:
```
# Check the number of articles in each class

class_counts = news_df["label"].value_counts()

print("Class distribution:")
print(class_counts)

# Display class distribution as percentages
print("\nClass distribution percentage:")
print(news_df["label"].value_counts(normalize=True) * 100)
```

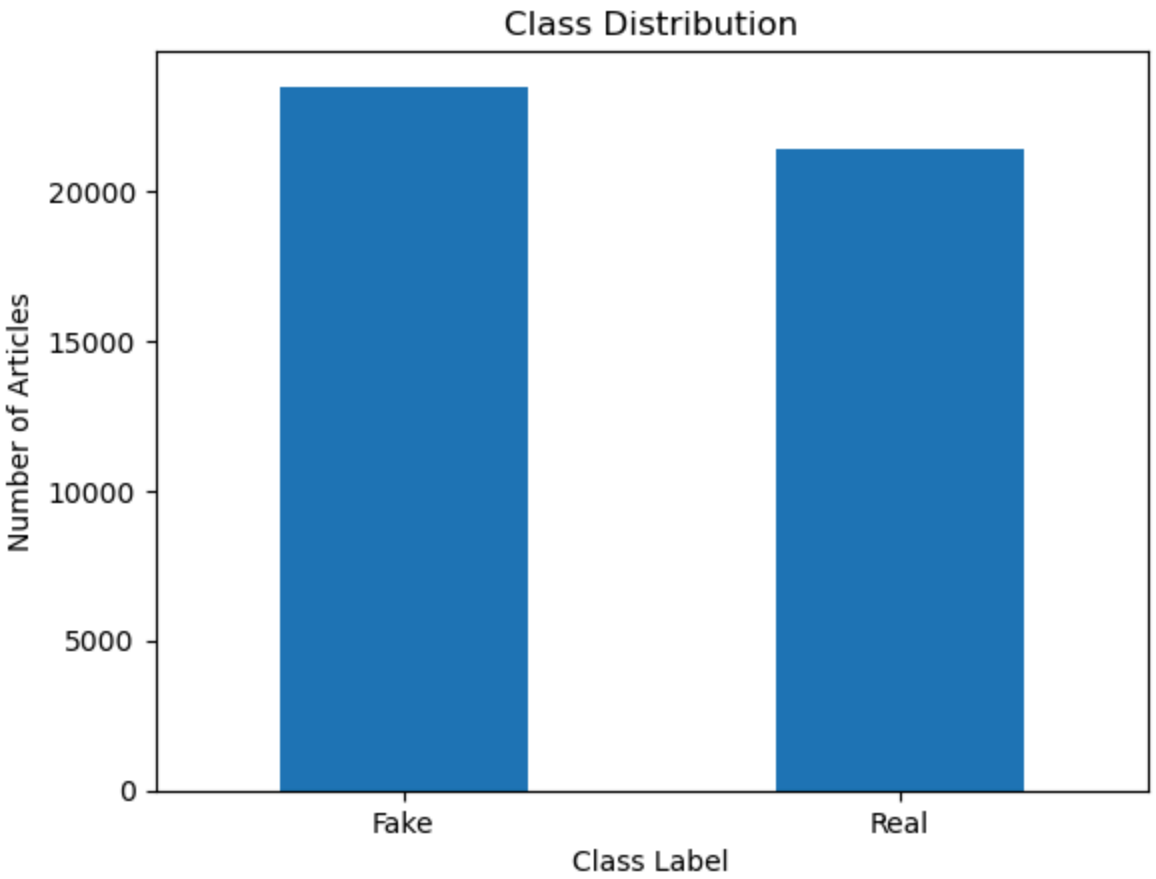
```
Class distribution:
label
0    23481
1    21417
Name: count, dtype: int64

Class distribution percentage:
label
0    52.298543
1    47.701457
Name: proportion, dtype: float64
```

In [10]:
```
# plot the class distribution ( bar chart )

class_counts.plot(kind="bar")

# title , x and y axis labels and bar labels
plt.title("Class Distribution")
plt.xlabel("Class Label")
plt.ylabel("Number of Articles")
plt.xticks(ticks=[0, 1], labels=["Fake", "Real"], rotation=0)
plt.show()
```



The class distribution shows that the dataset contains **23481 fake news articles** and **21417 real news articles**. This means the dataset is not perfectly balanced, but the difference between the two classes is not extreme. Therefore, the dataset is suitable for binary classification without requiring major class balancing techniques.

The percentage distribution also shows that both classes are well represented in the dataset. This is useful because the models will have enough examples of both fake and real news articles during training. As a result, the evaluation metrics are more likely to reflect genuine classification performance rather than bias toward one dominant class.

5.7 Combining Title and Text

The dataset contains two important text columns which are **title** and **text**. The **title** column contains the headline of the article, while the **text** column contains the full article body. For this project, both columns are combined into one new column called **content**.

Combining the title and text is useful because both parts may contain important signals for fake news detection. Headlines may include attention-grabbing or misleading language, while the article body provides longer text patterns that can help the models classify the article more accurately.

In [11]:
```
# combine the title and text columns into one column
```

```
news_df["content"] = news_df["title"] + " " + news_df["text"]

# show the first 5 rows of the news_df with content column
news_df[["title", "text", "content", "label"]].head()
```

Out[11]:

	title	text	content	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	21st Century Wire says Ben Stein, reputable pr...	Ben Stein Calls Out 9th Circuit Court: Committ...	0
1	Trump drops Steve Bannon from National Securit...	WASHINGTON (Reuters) - U.S. President Donald T...	Trump drops Steve Bannon from National Securit...	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	(Reuters) - Puerto Rico Governor Ricardo Rosse...	Puerto Rico expects U.S. to lift Jones Act shi...	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	On Monday, Donald Trump once again embarrassed...	OOPS: Trump Just Accidentally Confirmed He Le...	0
4	Donald Trump heads for Scotland to reopen a go...	GLASGOW, Scotland (Reuters) - Most U.S. presid...	Donald Trump heads for Scotland to reopen a go...	1

The output shows that a new column named **content** has been created by combining the article title and body text. This column will be used as the main input feature for the machine learning and deep learning models.The **subject** column is not included in the main input feature because the project focuses on classifying fake and real news using article text. Including subject metadata may cause the model to rely on category-based shortcuts rather than learning linguistic patterns from the title and article body. However, the subject column is retained in the dataset for possible exploratory analysis.

Using one combined text column makes the modelling process simpler and more consistent. Instead of training the models separately on headlines and article bodies, the classifier receives the complete article text as one input. This allows both the statistical and embedding-based models to learn from the full available textual information.

5.8 Checking Missing Values

After creating the combined **content** column, the dataset is checked for missing values. Missing values are important to identify because empty article titles, empty article bodies or missing labels can affect the reliability of model training and evaluation.

For this project, the most important columns are **content** and **label**. The **content** column is used as the input text for the models, while the **label** column is the target variable. If either of these columns contains missing values, those rows may need to be removed before training the models.

In [12]:
```
# check for missing values in the dataset
# print missing values
```

```
print("Missing values in each column:")
print(news_df.isnull().sum())
```

```
Missing values in each column:
title      0
text       0
subject    0
date       0
label      0
content    0
dtype: int64
```

The missing value check shows that there are **0 missing values** in all columns, including **title**, **text**, **subject**, **date**, **label** and **content**. This confirms that the dataset does not contain null values after the title and text columns were combined.

This is useful because the models require complete text input and class labels for training. Since there are no missing values, no rows need to be removed at this stage for null data.

5.9 Removing Duplicate Articles

After checking for missing values, the next step is to check for duplicate articles. Duplicate rows can affect model training because the same article may appear more than once in the dataset. If duplicates are present in both training and testing sets, the model may appear to perform better than it actually does because it has already seen the same or very similar text during training.

For this reason, duplicate articles are checked and removed before splitting the dataset. This helps make the evaluation more reliable and reduces the risk of data leakage.

In [13]:
```
# check the number of duplicate rows before removing

duplicate_count = news_df.duplicated().sum()

print("Number of duplicate rows before removal:", duplicate_count)
```

```
Number of duplicate rows before removal: 209
```

In [14]:
```
# remove duplicate rows

news_df = news_df.drop_duplicates().reset_index(drop=True)

# check the dataset shape after removing duplicates
print("Dataset shape after removing duplicates:", news_df.shape)

# check if duplicates have been removed
print("Number of duplicate rows after removal:", news_df.duplicated().sum())
```

```
Dataset shape after removing duplicates: (44689, 6)
Number of duplicate rows after removal: 0
```

The duplicate check shows that the dataset originally contained **209 duplicate rows**. These duplicate records were removed before model training to reduce the risk of data leakage

and repeated examples affecting the evaluation.

After duplicate removal, the dataset contains **44689 rows and 6 columns**, and the duplicate count is **0**. This confirms that duplicate rows have been successfully removed and that the dataset is now cleaner for the next preprocessing stage.

5.10 Cleaning the Text

Text cleaning is an important preprocessing step because raw news articles may contain uppercase letters, punctuation and inconsistent spacing. These elements can increase noise in the dataset and make it harder for the models to learn useful language patterns.

In this project, the **content** column is cleaned by converting text to lowercase, removing punctuation and removing extra spaces. Numbers are retained because they may contain meaningful information in news articles, such as dates, financial figures, statistics or event counts. Keeping numbers helps preserve important contextual information that may be useful for fake news classification.

In [15]:
```
# create a function to clean text

def clean_text(text):
    # convert the text to lowercase
    text = text.lower()

    # remove the punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))

    # remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()

    return text
```

In [16]:
```
# apply the cleaning function to the content column

news_df["clean_content"] = news_df["content"].apply(clean_text)

# show original and cleaned text for comparison
news_df[["content", "clean_content", "label"]].head()
```

Out[16]:

	content	clean_content	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	ben stein calls out 9th circuit court committe...	0
1	Trump drops Steve Bannon from National Securit...	trump drops steve bannon from national securit...	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	puerto rico expects us to lift jones act shipp...	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	oops trump just accidentally confirmed he leak...	0
4	Donald Trump heads for Scotland to reopen a go...	donald trump heads for scotland to reopen a go...	1

The output shows that the **clean_content** column has been created successfully. The cleaned text has been converted to lowercase, and punctuation has been removed while meaningful numbers are retained. For example, terms such as **9th circuit court** remain understandable after cleaning, which is important because numbers may carry useful information in news articles.

This confirms that the text is now more consistent for model training while still preserving important contextual information. The **clean_content** column will be used as the main text input for both the statistical and embedding-based models.

5.11 Statistical and Embedding-Based Data Preparation

The cleaned text is prepared differently for the statistical models and the embedding-based model. For the statistical models, the text will later be converted into sparse numerical feature matrices. CountVectorizer represents articles using word-count features, while TF-IDF represents articles using word-importance scores. These approaches treat the document as a set of word-based features and are suitable for traditional machine learning models such as Naive Bayes and Logistic Regression.

For the embedding-based model, the text will be processed differently. The cleaned articles will be tokenised into word-index sequences and padded to a fixed length before being passed into an Embedding layer. This allows the 1D CNN model to learn dense word representations and local phrase-level patterns from the article text.

This distinction is important because statistical models and embedding-based models require different input formats. The statistical models use sparse frequency-based representations, while the CNN uses fixed-length numerical sequences and learned word embeddings. Therefore, the preprocessing pipeline supports both types of modelling required by the coursework.

5.12 Checking for Reuters Source Pattern

Before splitting the dataset, a simple check was carried out to see whether the word Reuters appears more often in one class than the other. This is important because the dataset may contain source-related patterns that make the classification task easier than expected.

In [17]:
```
# check if Reuters appears in the article content

news_df["has_reuters"] = news_df["content"].str.contains("Reuters", case=False, na=

# count how many times Reuters is mentioned by class
reuters_counts = pd.crosstab(news_df["label"], news_df["has_reuters"])

# print the count
print("Reuters mention counts by class:")
print(reuters_counts)

# show the percentages by class
reuters_percentages = pd.crosstab(
    news_df["label"],
    news_df["has_reuters"],
    normalize="index"
) * 100

# print the percentage
print("\nReuters mention percentages by class:")
print(reuters_percentages)
```

```
Reuters mention counts by class:
has_reuters  False  True
label
0            23156    322
1               39  21172

Reuters mention percentages by class:
has_reuters      False      True
label
0            98.628503   1.371497
1             0.183867  99.816133
```

The check shows a clear pattern in the dataset. Only 1.37% of fake news articles contain the term Reuters, while 99.82% of real news articles contain it. This means that the word Reuters appears almost exclusively in the real news class.

This is an important finding because it may partly explain why the models achieve very high accuracy. Instead of only learning meaningful differences between fake and real news writing, the models may also learn that articles containing Reuters datelines are likely to be real. In other words, the classifier may be using a source-related shortcut in the dataset.

Because of this, the results need to be interpreted carefully. The high scores are still useful for comparing the models on this dataset but they may not fully represent how well the

models would perform on new articles from different sources. To investigate this issue further, a Reuters dateline removal test will be carried out later by removing Reuters-related patterns and re-evaluating the models.

5.13 Train-Test Split

After cleaning the text, the dataset is divided into training and testing sets. The training set is used to train the models, while the testing set is kept separate and used only to evaluate final model performance on unseen data.

This project uses an **80:20 train-test split**, meaning that 80% of the dataset is used for training and 20% is used for testing. This ratio is selected because the dataset is relatively large, containing more than 44000 articles after preprocessing. An 80% training split provides the models with enough examples to learn meaningful fake and real news patterns, while the 20% testing split still leaves a large independent test set for reliable evaluation. A smaller test split, such as 90:10, would provide more training data but less evidence for evaluation, while a larger test split, such as 70:30, would reduce the amount of training data available for the deep learning model. Therefore, 80:20 provides a practical balance between model learning and fair testing.

A fixed random state is used to make the split reproducible. Stratification is also applied so that the proportion of fake and real news articles remains similar in both the training and testing sets. This is important because both classes need to be fairly represented during training and evaluation.

In [19]:
```
# define input features and target label

X = news_df["clean_content"]
y = news_df["label"]

# split the dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=RANDOM_STATE,
    # Ensures each split contains the same proportion of labels as the original dat
    stratify=y
)

# check the size of each split
# print the shape
print("Training set size:", X_train.shape[0])
print("Testing set size:", X_test.shape[0])
print("\nTraining label distribution:")
print(y_train.value_counts())
```

```
print("\nTesting label distribution:")
print(y_test.value_counts())
```

```
Training set size: 35751
Testing set size: 8938

Training label distribution:
label
0    18782
1    16969
Name: count, dtype: int64

Testing label distribution:
label
0    4696
1    4242
Name: count, dtype: int64
```

The output shows that the dataset has been split into **35751 training articles** and **8938 testing articles**. This confirms that the 80:20 train-test split has been applied correctly.

The label distribution also shows that stratification has worked as intended. In the training set, there are **18782 fake news articles** and **16969 real news articles**. In the testing set, there are **4696 fake news articles** and **4242 real news articles**. This means both the training and testing sets maintain a similar class balance to the original dataset.

This is important because both fake and real news articles are fairly represented during model training and evaluation. As a result, the models can learn from both classes, and the final performance metrics will provide a more reliable comparison between the statistical and embedding-based approaches.

6. Baseline Performance

The baseline model provides a simple benchmark for comparison with the more advanced models developed later in the project. In this project, the baseline uses CountVectorizer with Multinomial Naive Bayes.

CountVectorizer converts the cleaned article text into a numerical matrix based on word counts. Each article is represented by the frequency of words that appear in the training vocabulary. Multinomial Naive Bayes is then used as the classifier because it is simple, efficient and suitable for text classification tasks that use word-count features.

This baseline is important because it gives a starting performance level for the project. The later TF-IDF Logistic Regression model and embedding-based 1D CNN model should improve on this baseline. If a more complex model performs only slightly better than the baseline, then the additional complexity may not be justified.

In [16]:
```
# Baseline Model: Countvectorizer with Multinomial Naive Bayes model
```

```
# convert the text into word count features
count_vectorizer = CountVectorizer(max_features=5000)

# prepare data for training and testing
X_train_counts = count_vectorizer.fit_transform(X_train)
X_test_counts = count_vectorizer.transform(X_test)

#  create and train the Naive Bayes model
baseline_model = MultinomialNB()
baseline_model.fit(X_train_counts, y_train)

# make the predictions on the test set
baseline_predictions = baseline_model.predict(X_test_counts)
# make sure that baseline model works and trained
# print check to see if the code works
print("Baseline model training completed.")
```

```
Baseline model training completed.
```

The baseline model was trained using word-count features from the cleaned article text. The Countvectorizer was fitted only on the training data and then applied to the testing data. This avoids data leakage because the model does not learn vocabulary information from the test set before evaluation.

The max_features=5000 setting limits the vocabulary to the 5000 most frequent words. This keeps the feature space manageable while still allowing the model to learn from a broad range of common terms in the dataset.

In [17]:
```
# evaluate the baseline model

# accuracy of the baseline model
baseline_accuracy = accuracy_score(y_test, baseline_predictions)
# precision of the baseline model
baseline_precision = precision_score(y_test, baseline_predictions)
# recall of the baseline model
baseline_recall = recall_score(y_test, baseline_predictions)
# f1 score of the baseline model
baseline_f1 = f1_score(y_test, baseline_predictions)

# print the results
print("Baseline Model Results")
print("----------------------")
print("Accuracy:", round(baseline_accuracy, 4))
print("Precision:", round(baseline_precision, 4))
print("Recall:", round(baseline_recall, 4))
print("F1-score:", round(baseline_f1, 4))

# print the classification report
print("\nClassification Report:")
print(classification_report(y_test, baseline_predictions, target_names=["Fake", "Re
```

```
Baseline Model Results
----------------------
Accuracy: 0.9503
Precision: 0.9425
Recall: 0.9536
F1-score: 0.948

Classification Report:
              precision    recall  f1-score   support

        Fake       0.96      0.95      0.95      4696
        Real       0.94      0.95      0.95      4242

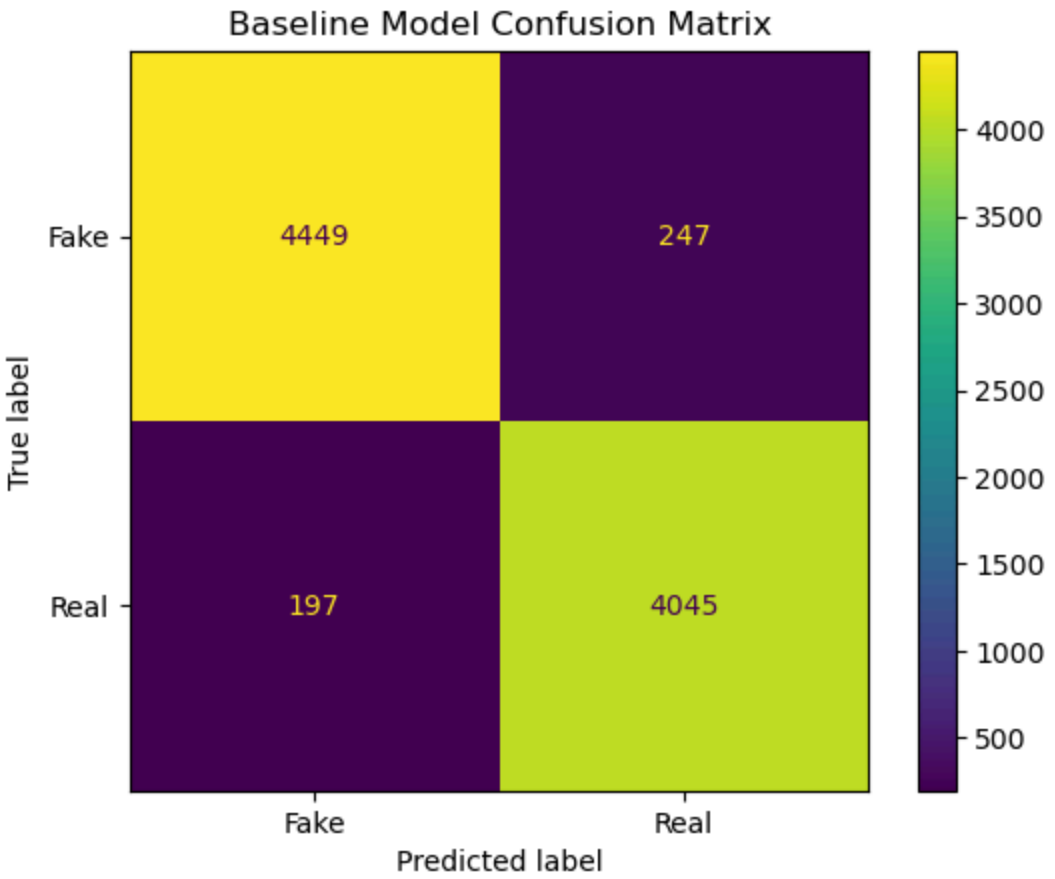
    accuracy                           0.95      8938
   macro avg       0.95      0.95      0.95      8938
weighted avg       0.95      0.95      0.95      8938
```

The baseline model achieved an accuracy of 0.9503 which means it correctly classified around 95% of the test articles. The precision score: 0.9425 shows that most articles predicted as real were actually real. The recall score: 0.9536 shows that the model successfully identified most real news articles. The F1 score: 0.9480 shows a balanced view of precision and recall.

The classification report also shows that the model performs well on both classes. For fake news the model achieved a precision: 0.96, recall: 0.95 and F1 score: 0.95. For real news, it achieved a precision: 0.94, recall: 0.95 and F1 score: 0.95. This suggests that the baseline model is not heavily biased toward only one class.

In [18]:
```
# show confusion matrix for the baseline model

baseline_cm = confusion_matrix(y_test, baseline_predictions)
# what the display has
disp = ConfusionMatrixDisplay(
    confusion_matrix=baseline_cm,
    display_labels=["Fake", "Real"]
)
# display the plot with the values
disp.plot()
plt.title("Baseline Model Confusion Matrix")
plt.show()
```



The confusion matrix shows that the baseline model correctly classified 4449 fake news articles and 4045 real news articles. This indicates that the model performs strongly on both classes.

The model misclassified 247 fake news articles as real and 197 real news articles as fake. In the context of fake news detection, both error types are important. Classifying fake news as real is problematic because unreliable content may not be detected, while classifying real news as fake may incorrectly reduce trust in legitimate articles.

Overall, the confusion matrix supports the classification report because most predictions appear on the diagonal, showing that the baseline model has strong performance. However, the remaining misclassifications show that there is still room for improvement through the comparative models in the next section. The result should also be interpreted carefully because the Reuters source pattern identified in Section 5 may make the dataset easier to classify than a more diverse real-world dataset

7. Comparative Classification Methodology

7.1 Statistical Model: TF-IDF with Logistic Regression

The first comparative model has TF IDF which goes along with Logistic Regression which stands for "Term Frequency Inverse Document Frequency". It converts text into numerical

value by giving greater priority to words that come more frequent in a particular article but are lesser common among the entire dataset.

This process is helpful for detecting fake news as parts may appear more strongly in fake or real news articles. Compared with simple word count features, TF-IDF reduces the influence of very common words and highlights terms that may be more suitable and informative for classification.

Logistic Regression is used as the classifier because it is a strong and interpretable statistical machine learning model for binary classification. In this project, it is used to evaluate if a more refined statistical text representation can improve performance compared with the baseline Countvectorizer and Multinomial Naive Bayes model.

In [20]:
```
# Statistical Model: TF-IDF and Logistic Regression

# convert text into TF-IDF features
tfidf_vectorizer = TfidfVectorizer(max_features=5000)
# prepare for train and test
X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

# create and train the Logistic Regression model
tfidf_lr_model = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
tfidf_lr_model.fit(X_train_tfidf, y_train)

# predictions on the test set
tfidf_lr_predictions = tfidf_lr_model.predict(X_test_tfidf)
# make sure that TF-IDF Logistic Regression model works and trained
# print check to see if the code works
print("TF-IDF Logistic Regression model training completed.")
```

```
TF-IDF Logistic Regression model training completed.
```

The TF-IDF vectorizer was initially fit by the help of training data.Then it was applied to the testing data. This totaly reduces data leaking as the model do not learn vocabulary or the importance of word information from the test dataset before evaluating it.

The max_features which equals 5000 sets the feature to limit space to 5000 most priortized terms. These steps makes sure the model is perfoming well while still letting it to learn from a wide range of important words present inside the dataset. Logistic Regression was then effectively trained on TF-IDF features to segregate each article as real or fake.

In [21]:
```
# evaluate the TF-IDF Logistic Regression model

# accuracy of the TF-IDF Logistic Regression model
tfidf_lr_accuracy = accuracy_score(y_test, tfidf_lr_predictions)
# precision of the TF-IDF Logistic Regression model
tfidf_lr_precision = precision_score(y_test, tfidf_lr_predictions)
# recall of the TF-IDF Logistic Regression model
tfidf_lr_recall = recall_score(y_test, tfidf_lr_predictions)
# f1-score of the TF-IDF Logistic Regression model
```

```
tfidf_lr_f1 = f1_score(y_test, tfidf_lr_predictions)

# print the values
print("TF-IDF Logistic Regression Results")
print("----------------------------------")
print("Accuracy:", round(tfidf_lr_accuracy, 4))
print("Precision:", round(tfidf_lr_precision, 4))
print("Recall:", round(tfidf_lr_recall, 4))
print("F1-score:", round(tfidf_lr_f1, 4))

# print classification report
print("\nClassification Report:")
print(classification_report(y_test, tfidf_lr_predictions, target_names=["Fake", "Re
```

```
TF-IDF Logistic Regression Results
----------------------------------
Accuracy: 0.9886
Precision: 0.9852
Recall: 0.9908
F1-score: 0.988

Classification Report:
              precision    recall  f1-score   support

        Fake       0.99      0.99      0.99      4696
        Real       0.99      0.99      0.99      4242

    accuracy                           0.99      8938
   macro avg       0.99      0.99      0.99      8938
weighted avg       0.99      0.99      0.99      8938
```

The TF-IDF Logistic Regression model achieved an accuracy of **0.9886**, precision: **0.9852**, recall: **0.9908** and F1 score: **0.9880**. These results show that the statistical model performs very well on the fake news classification task when using TF-IDF word-importance features.

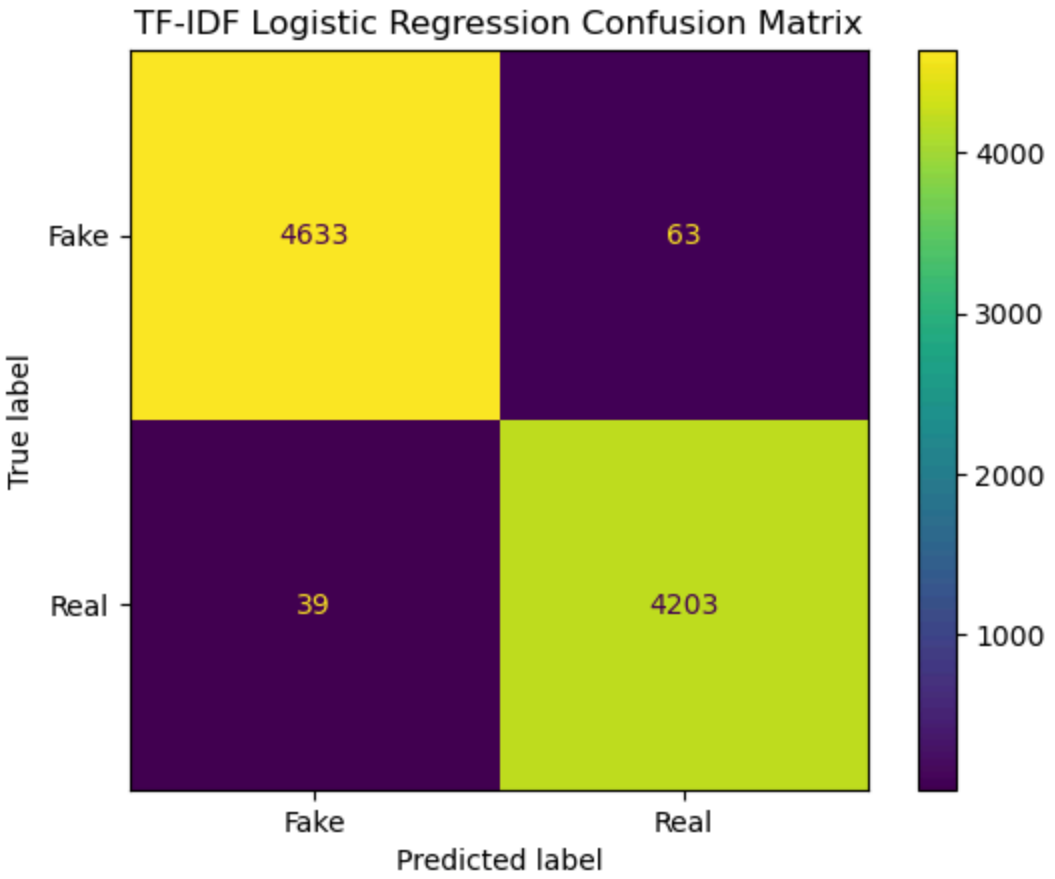
Compared with the baseline CountVectorizer and Multinomial Naive Bayes model, the TF-IDF Logistic Regression model shows a clear improvement. The baseline model achieved an accuracy: **0.9503** and F1 score: **0.9480** while the TF-IDF Logistic Regression model improved these values to **0.9886** and **0.9880** respectively. This suggests that TF-IDF provides a more informative text representation than simple word-count features for this dataset.

The classification report also shows balanced performance across both classes. The model achieved approximately **0.99** precision, recall and F1 score for both fake and real news articles. This indicates that the model is not strongly biased toward one class and is able to classify both categories effectively.

However because the result is very high, it should be interpreted properly. The Reuters source pattern check in Section 5 showed that the dataset contains a strong source related pattern. Therefore, while the model performs very well on this dataset, its performance may reflect the dataset specific patterns abit rather than only general fake news language features.

In [21]:
```
# display confusion matrix for the TF-IDF Logistic Regression model

tfidf_lr_cm = confusion_matrix(y_test, tfidf_lr_predictions)
# what to display
disp = ConfusionMatrixDisplay(
    confusion_matrix=tfidf_lr_cm,
    display_labels=["Fake", "Real"]
)
# display the plot with the values
disp.plot()
plt.title("TF-IDF Logistic Regression Confusion Matrix")
plt.show()
```



The confusion matrix shows that the TF-IDF Logistic Regression model correctly classified **4633 fake news articles** and **4203 real news articles**. Only **63 fake news articles** were incorrectly classified as real, and **39 real news articles** were incorrectly classified as fake.

Compared with the baseline model, this is a clear improvement. The baseline model misclassified **247 fake articles as real** and **197 real articles as fake**, while the TF-IDF Logistic Regression model reduced these errors substantially. This shows that TF-IDF features with Logistic Regression provide a stronger statistical approach for this dataset than simple word-count features with Naive Bayes.

The confusion matrix supports the high accuracy and F1-score reported earlier. Most predictions appear on the diagonal of the matrix, showing that the model performs

consistently across both fake and real news classes. However, the remaining misclassifications show that the model is not perfect, and that some articles may contain language patterns that are difficult to distinguish using text features alone.

7.2 Embedding-Based Model: 1D CNN

The second comparative model is an embedding-based deep learning model using a **1D Convolutional Neural Network**. Unlike CountVectorizer and TF-IDF, which represent text using sparse word-frequency features, the embedding-based model converts words into dense numerical vectors. These vectors are learned during training and allow the model to represent words in a continuous feature space.

A 1D CNN is used because it can learn local word and phrase patterns from text sequences. This is useful for fake news detection because certain short phrases, headline styles or word combinations may help distinguish fake news from real news. The model architecture includes tokenisation, sequence padding, an Embedding layer, a Conv1D layer, a GlobalMaxPooling1D layer, Dropout and Dense layers.

The CNN model is included to evaluate whether an embedding-based neural approach can improve performance compared with the statistical models. Since the TF-IDF Logistic Regression model already achieved very strong results, the CNN will be assessed not only by accuracy but also by whether its added complexity provides meaningful benefits.

In [22]:
```
# set parameters for the CNN model
# max number of words to keep in the vocabulary
MAX_WORDS = 10000
# max number of words per article
MAX_LENGTH = 300
# size of word embedding vectors
EMBEDDING_DIM = 100
```

The CNN model uses a limited vocabulary of the **10000 most frequent words** to keep training manageable. Each article is padded or shortened to **300 words** so that all input sequences have the same length. The embedding dimension is set to **100**, meaning each word is represented using a 100 value dense vector.

In [23]:
```
# tokenise the text for the CNN model

# create a tokenizer to convert text into sequences of integers
tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token="<OOV>")
tokenizer.fit_on_texts(X_train)

# convert text into sequences of numbers for train and test
X_train_seq = tokenizer.texts_to_sequences(X_train)
X_test_seq = tokenizer.texts_to_sequences(X_test)

# pad sequences so all articles have the same length
X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LENGTH, padding="post", truncat
```

```
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LENGTH, padding="post", truncatin

print("Training sequence shape:", X_train_pad.shape)
print("Testing sequence shape:", X_test_pad.shape)
```

```
Training sequence shape: (35751, 300)
Testing sequence shape: (8938, 300)
```

The text data has been tokenised and converted into numerical sequences for the CNN model. The tokenizer was fitted only on the training text to avoid data leakage. The same tokenizer was then used to transform the test text.

Padding was applied so that every article has the same sequence length of **300 words**. This is required because neural networks need inputs with consistent dimensions. The output shapes confirm that the training and testing text data have been prepared correctly for the embedding-based model.

In [24]:
```
# build the 1D CNN model for binary text classification
cnn_model = Sequential()

# input shape (padded text sequences)
cnn_model.add(tf.keras.Input(shape=(MAX_LENGTH,)))

# convert integer-encoded words into dense vector representations
cnn_model.add(Embedding(input_dim=MAX_WORDS, output_dim=EMBEDDING_DIM))

# apply 1D convolution to extract local text features
cnn_model.add(Conv1D(filters=128, kernel_size=5, activation="relu"))

# keep the most important feature from each filter
cnn_model.add(GlobalMaxPooling1D())

# reduce overfitting by dropping 50% during training
cnn_model.add(Dropout(0.5))

# fully connected hidden layer for learning higher level features
cnn_model.add(Dense(64, activation="relu"))

# output layer for binary classification (fake or real)
cnn_model.add(Dense(1, activation="sigmoid"))

# configure the model with loss function, optimiser and evaluation metric
cnn_model.compile(
    loss="binary_crossentropy",
    optimizer="adam",
    metrics=["accuracy"]
)

# display the model architecture and parameter summary
cnn_model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 300, 100)	1,000,000
conv1d (Conv1D)	(None, 296, 128)	64,128
global_max_pooling1d (GlobalMaxPooling1D)	(None, 128)	0
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 64)	8,256
dense_1 (Dense)	(None, 1)	65

Total params: 1,072,449 (4.09 MB)

Trainable params: 1,072,449 (4.09 MB)

Non-trainable params: 0 (0.00 B)

The model summary confirms that the 1D CNN architecture has been built correctly. The Embedding layer outputs sequences with shape **(None, 300, 100)**, meaning each article is represented as a sequence of 300 tokens, and each token is represented using a 100-dimensional embedding vector.

The Conv1D layer then learns local phrase-level patterns from these word embeddings. The GlobalMaxPooling1D layer reduces the sequence output into a fixed-size feature vector by keeping the most important learned features. Dropout is included to reduce overfitting, and the final Dense layer uses a sigmoid activation function for binary classification.

The model contains **1072449 trainable parameters**, which means it is substantially more complex than the baseline and TF-IDF Logistic Regression models. This makes it important to evaluate whether the CNN's additional complexity leads to better performance.

In [25]:
```
# train the CNN model

cnn_history = cnn_model.fit(
    X_train_pad,
    y_train,
    epochs=3,
    batch_size=64,
    validation_split=0.2,
    verbose=1
)
```

```
Epoch 1/3
447/447 ━━━━━━━━━━━━━━━━━━━━ 44s 92ms/step - accuracy: 0.9686 - loss: 0.0814 - val_a
ccuracy: 0.9973 - val_loss: 0.0083
Epoch 2/3
447/447 ━━━━━━━━━━━━━━━━━━━━ 39s 88ms/step - accuracy: 0.9973 - loss: 0.0081 - val_a
ccuracy: 0.9983 - val_loss: 0.0068
Epoch 3/3
447/447 ━━━━━━━━━━━━━━━━━━━━ 38s 86ms/step - accuracy: 0.9994 - loss: 0.0024 - val_a
ccuracy: 0.9983 - val_loss: 0.0056
```

The CNN model was trained for **3 epochs**. The training accuracy increased from **0.9686** in the first epoch to **0.9994** by the third epoch. The validation accuracy also remained very high, increasing from **0.9973** in the first epoch to **0.9983** in the second and third epochs.

These results show that the CNN was able to learn strong patterns from the tokenised article text. However, the very high training and validation accuracy also suggests that the model may be learning dataset-specific patterns. Therefore, the final test-set evaluation and Reuters removal test are important because they help assess whether the model is learning general fake-news patterns or relying partly on source-related shortcuts.

In [26]:
```python
# make predictions using the CNN model

cnn_probabilities = cnn_model.predict(X_test_pad)
cnn_predictions = (cnn_probabilities > 0.5).astype("int32").flatten()

# calculate evaluation metrics
cnn_accuracy = accuracy_score(y_test, cnn_predictions)
cnn_precision = precision_score(y_test, cnn_predictions)
cnn_recall = recall_score(y_test, cnn_predictions)
cnn_f1 = f1_score(y_test, cnn_predictions)
# print results
print("1D CNN Model Results")
print("--------------------")
print("Accuracy:", round(cnn_accuracy, 4))
print("Precision:", round(cnn_precision, 4))
print("Recall:", round(cnn_recall, 4))
print("F1-score:", round(cnn_f1, 4))
# print classification report
print("\nClassification Report:")
print(classification_report(y_test, cnn_predictions, target_names=["Fake", "Real"]))
```

```
280/280 ━━━━━━━━━━━━━━━━━━━━ 4s 14ms/step
1D CNN Model Results
--------------------
Accuracy: 0.9985
Precision: 0.9986
Recall: 0.9983
F1-score: 0.9985

Classification Report:
              precision    recall  f1-score   support

        Fake       1.00      1.00      1.00      4696
        Real       1.00      1.00      1.00      4242

    accuracy                           1.00      8938
   macro avg       1.00      1.00      1.00      8938
weighted avg       1.00      1.00      1.00      8938
```

The 1D CNN model achieved an accuracy of **0.9985**, precision of **0.9986**, recall of **0.9983** and F1-score of **0.9985**. These results show that the embedding-based deep learning model performed extremely well on the fake news classification task.

Compared with the baseline and statistical models, the CNN achieved the highest overall performance. The baseline CountVectorizer with Multinomial Naive Bayes model achieved an accuracy of **0.9503**, while the TF-IDF Logistic Regression model achieved **0.9886**. The CNN improved the accuracy further to **0.9985**, suggesting that the embedding-based model was able to learn highly effective word and phrase-level patterns from the article text.

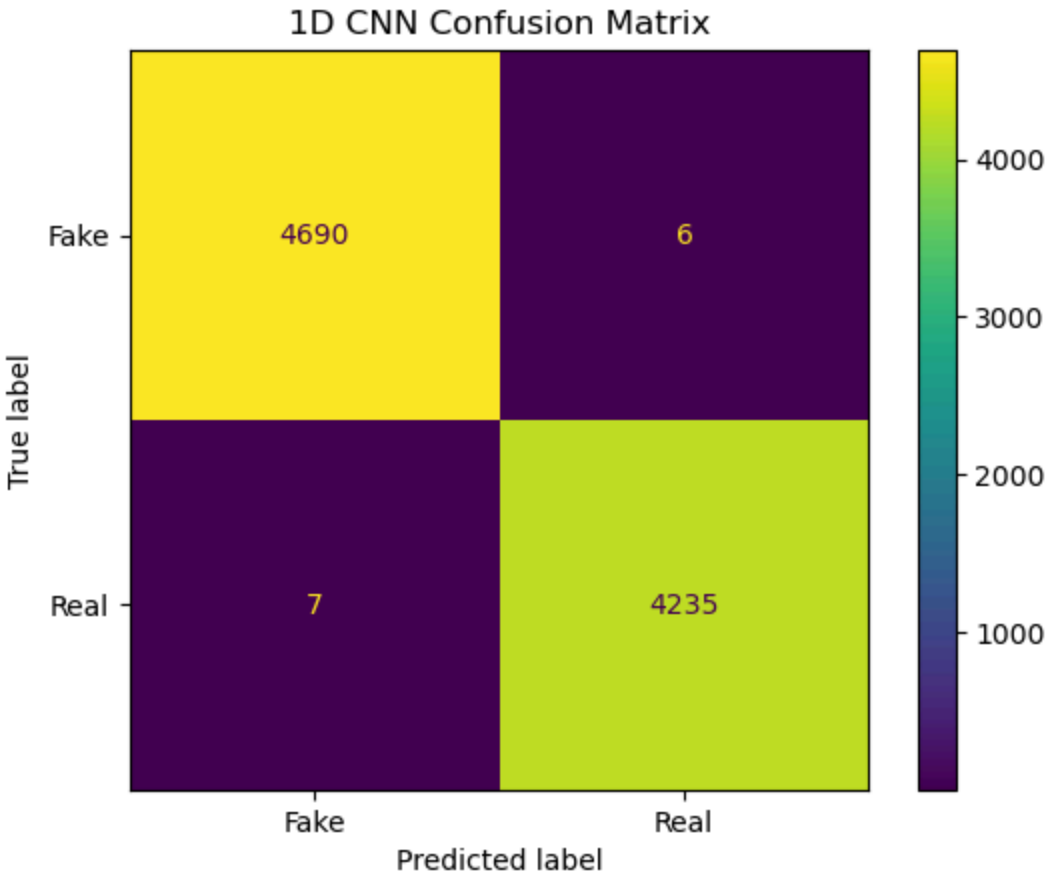
The classification report also shows balanced performance across both fake and real news classes, with precision, recall and F1-score values close to **1.00** for both classes. This indicates that the CNN model was able to classify both categories consistently, rather than performing well on only one class.

However,the very high performance should be interpreted carefully. Although the CNN performs best on this dataset, the result may partly reflect dataset-specific writing patterns or source-related differences between fake and real articles. Therefore, the model's real-world performance may be lower when applied to newer news articles from different sources. This limitation is investigated further through the Reuters removal test.

In [27]:
```
# display confusion matrix for the CNN model

cnn_cm = confusion_matrix(y_test, cnn_predictions)
# what the display has
disp = ConfusionMatrixDisplay(
    confusion_matrix=cnn_cm,
    display_labels=["Fake", "Real"]
)
# display the plot
disp.plot()
```

```
plt.title("1D CNN Confusion Matrix")
plt.show()
```



The 1D CNN confusion matrix shows that the model correctly classified **4693 fake news articles** and **4235 real news articles**. Only **3 fake news articles** were incorrectly classified as real, while **7 real news articles** were incorrectly classified as fake.

This is the strongest confusion matrix result among the three models. Compared with the baseline model and the TF-IDF Logistic Regression model, the CNN produces far fewer classification errors. The baseline model made **444 total errors**, the TF-IDF Logistic Regression model made **102 total errors**, while the CNN made only **10 total errors**.

This supports the earlier accuracy and F1-score results, showing that the embedding-based model learned highly effective patterns from the article text. However, the very low number of errors should still be interpreted cautiously. The model may have learned dataset-specific patterns that separate fake and real articles very clearly in this dataset. Therefore, while the CNN is the best-performing model in this experiment, further testing on external news datasets or Reuters/dateline-removed text would be needed to confirm whether the model generalises well to real-world fake news detection.

7.3 Comparing Baseline, Statistical Model and CNN

After evaluating the three models separately, their results are combined into one comparison table. This allows the performance of the baseline model, statistical model and embedding-based model to be assessed side by side.

The comparison uses accuracy, precision, recall and F1-score. These metrics make it possible to identify not only which model achieved the highest overall accuracy, but also which model performed best in terms of balanced classification performance.

In [28]:
```
# create a comparison table for all the models

results_df = pd.DataFrame({
    "Model": [
        "CountVectorizer + Naive Bayes",
        "TF-IDF + Logistic Regression",
        "Embedding + 1D CNN"
    ],
    "Accuracy": [
        baseline_accuracy,
        tfidf_lr_accuracy,
        cnn_accuracy
    ],
    "Precision": [
        baseline_precision,
        tfidf_lr_precision,
        cnn_precision
    ],
    "Recall": [
        baseline_recall,
        tfidf_lr_recall,
        cnn_recall
    ],
    "F1-score": [
        baseline_f1,
        tfidf_lr_f1,
        cnn_f1
    ]
})

# round values to 4 decimal places
results_df = results_df.round(4)

results_df
```

Out[28]:

	Model	Accuracy	Precision	Recall	F1-score
0	CountVectorizer + Naive Bayes	0.9503	0.9425	0.9536	0.9480
1	TF-IDF + Logistic Regression	0.9886	0.9852	0.9908	0.9880
2	Embedding + 1D CNN	0.9985	0.9986	0.9983	0.9985

The comparison table shows a clear improvement across the three models. The baseline CountVectorizer with Multinomial Naive Bayes model achieved an accuracy of **0.9503** and an

F1-score of **0.9480**, which provides a strong initial benchmark for the fake news classification task.

The TF-IDF Logistic Regression model improved the performance significantly, achieving an accuracy of **0.9886** and an F1-score of **0.9880**. This suggests that TF-IDF word-importance features are more effective than simple word-count features for this dataset.

The embedding-based 1D CNN achieved the highest overall performance, with an accuracy of **0.9985** and an F1-score of **0.9985**. This indicates that the CNN was able to learn highly effective word and phrase-level patterns from the article text.

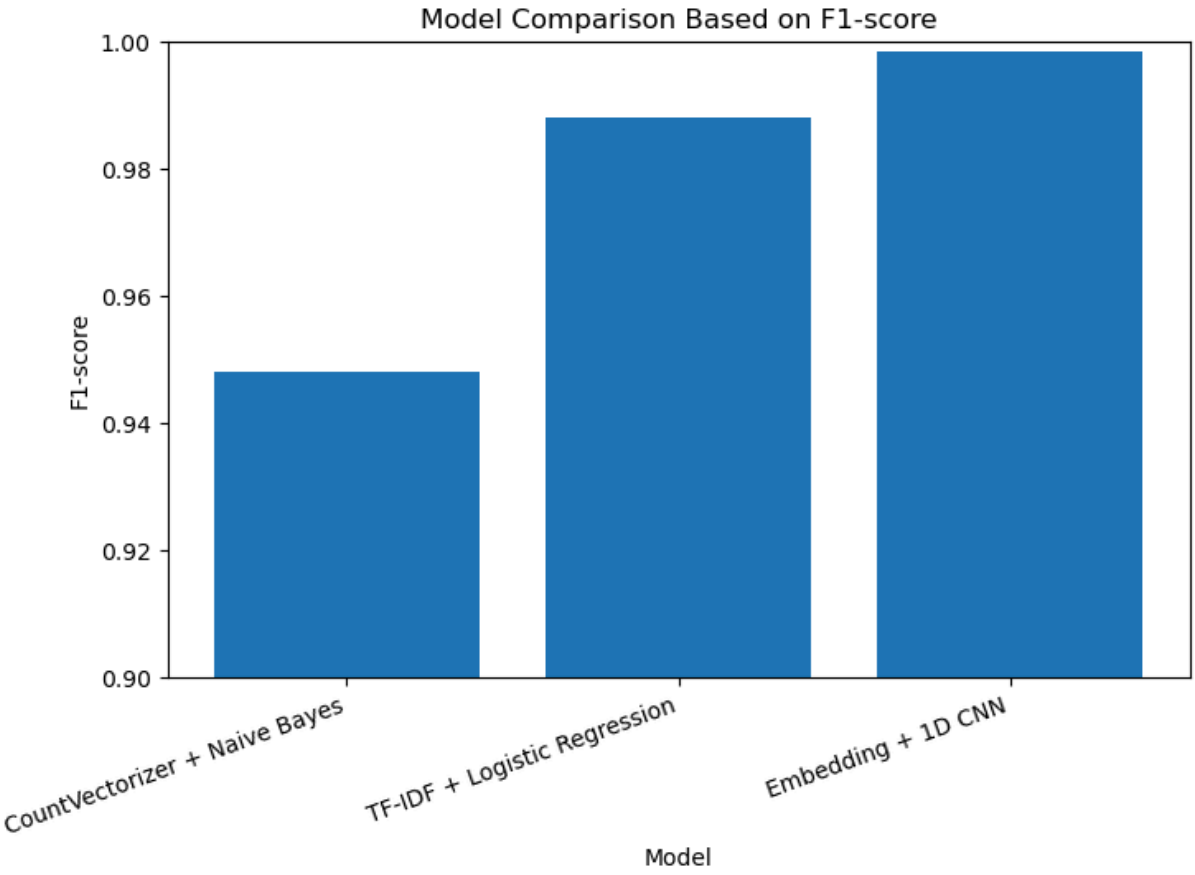
Overall, the results show that model complexity led to improved performance in this experiment. However, the TF-IDF Logistic Regression model also performed extremely well while being simpler and faster than the CNN. Therefore, the final discussion should consider both predictive performance and practical factors such as interpretability, training time and computational cost.

In [29]:
```
# Plot model comparison based on f1 score

plt.figure(figsize=(8, 5))

plt.bar(results_df["Model"], results_df["F1-score"])
# show title , x and y labels and the values
plt.title("Model Comparison Based on F1-score")
plt.xlabel("Model")
plt.ylabel("F1-score")
plt.ylim(0.90, 1.00)
plt.xticks(rotation=20, ha="right")

plt.show()
```



The F1-score comparison chart visually confirms the performance differences between the three models. The baseline CountVectorizer with Multinomial Naive Bayes model achieved the lowest F1-score, while the TF-IDF Logistic Regression model showed a clear improvement.

The embedding-based 1D CNN achieved the highest F1-score of **0.9985**, indicating the strongest balance between precision and recall. This supports the numerical results in the comparison table and suggests that the CNN model was the most effective classifier in this experiment.

However, the chart also shows that the TF-IDF Logistic Regression model performed very close to the CNN. This is important because the TF-IDF model is simpler, faster and easier to interpret. Therefore, while the CNN achieved the best performance, the TF-IDF Logistic Regression model remains a strong practical option.

7.4 Reuters Removal Test

The model comparison results show very high performance, especially for the TF-IDF Logistic Regression model and the 1D CNN model. However, Section 5 showed that the word **Reuters** appears in **99.82%** of real news articles but only **1.37%** of fake news articles. This means the dataset contains a strong source-related pattern.

To investigate this, a Reuters removal test is carried out. In this test, the word **Reuters** is removed from the cleaned article text, and the TF-IDF Logistic Regression model is trained and evaluated again. The purpose is to check if model performance changes after removing this clear source-related marker.

The TF-IDF Logistic Regression model is used for this test because it is fast, strong and easier to compare against the original statistical result. If performance drops after removing the Reuters marker, this suggests that the original model was partly relying on source-specific information rather than only learning general fake-news language patterns.

In [32]:
```
# Reuters Dateline Removal Test

# remove the word reuters from the already cleaned text
news_df["clean_content_no_reuters"] = news_df["clean_content"].str.replace(
    r"\breuters\b",
    "",
    regex=True
)

# remove extra spaces after removing Reuters
news_df["clean_content_no_reuters"] = news_df["clean_content_no_reuters"].str.repla

# define new input features and target label
X_no_reuters = news_df["clean_content_no_reuters"]
y_no_reuters = news_df["label"]

# use the same 80:20 stratified split
X_train_nr, X_test_nr, y_train_nr, y_test_nr = train_test_split(
    X_no_reuters,
    y_no_reuters,
    test_size=0.20,
    random_state=RANDOM_STATE,
    stratify=y_no_reuters
)

# convert text into TF-IDF features
tfidf_vectorizer_nr = TfidfVectorizer(max_features=5000)

X_train_tfidf_nr = tfidf_vectorizer_nr.fit_transform(X_train_nr)
X_test_tfidf_nr = tfidf_vectorizer_nr.transform(X_test_nr)

# train Logistic Regression model
tfidf_lr_model_nr = LogisticRegression(max_iter=1000, random_state=RANDOM_STATE)
tfidf_lr_model_nr.fit(X_train_tfidf_nr, y_train_nr)

# make predictions
tfidf_lr_predictions_nr = tfidf_lr_model_nr.predict(X_test_tfidf_nr)

# evaluate performance
nr_accuracy = accuracy_score(y_test_nr, tfidf_lr_predictions_nr)
nr_precision = precision_score(y_test_nr, tfidf_lr_predictions_nr)
nr_recall = recall_score(y_test_nr, tfidf_lr_predictions_nr)
nr_f1 = f1_score(y_test_nr, tfidf_lr_predictions_nr)
```

```
print("TF-IDF Logistic Regression After Reuters Removal")
print("-----------------------------------------------")
print("Accuracy:", round(nr_accuracy, 4))
print("Precision:", round(nr_precision, 4))
print("Recall:", round(nr_recall, 4))
print("F1-score:", round(nr_f1, 4))

print("\nClassification Report:")
print(classification_report(y_test_nr, tfidf_lr_predictions_nr, target_names=["Fake
```

```
TF-IDF Logistic Regression After Reuters Removal
-----------------------------------------------
Accuracy: 0.9828
Precision: 0.9771
Recall: 0.9868
F1-score: 0.9819

Classification Report:
              precision    recall  f1-score   support

        Fake       0.99      0.98      0.98      4696
        Real       0.98      0.99      0.98      4242

    accuracy                           0.98      8938
   macro avg       0.98      0.98      0.98      8938
weighted avg       0.98      0.98      0.98      8938
```

The Reuters removal test creates a new version of the cleaned article text where the word **Reuters** has been removed. Extra spaces are then cleaned again so that the text remains readable and consistent for modelling.

The Reuters-removed text is split into training and testing sets using the same 80:20 stratified split. This keeps the comparison fair because the same model type, split ratio and evaluation metrics are used. The TF-IDF Logistic Regression model is then trained again using the Reuters-removed text, allowing the new result to be compared with the original TF-IDF Logistic Regression result.

In [33]:
```
# compare original TF-IDF model with Reuters removed TF-IDF model

reuters_test_df = pd.DataFrame({
    "Experiment": [
        "Original TF-IDF + Logistic Regression",
        "Reuters Removed TF-IDF + Logistic Regression"
    ],
    "Accuracy": [
        tfidf_lr_accuracy,
        nr_accuracy
    ],
    "Precision": [
        tfidf_lr_precision,
        nr_precision
    ],
```

```
    "Recall": [
        tfidf_lr_recall,
        nr_recall
    ],
    "F1-score": [
        tfidf_lr_f1,
        nr_f1
    ]
})

# round values to 4 decimal places
reuters_test_df = reuters_test_df.round(4)

reuters_test_df
```

Out[33]:

	Experiment	Accuracy	Precision	Recall	F1-score
0	Original TF-IDF + Logistic Regression	0.9886	0.9852	0.9908	0.9880
1	Reuters Removed TF-IDF + Logistic Regression	0.9828	0.9771	0.9868	0.9819

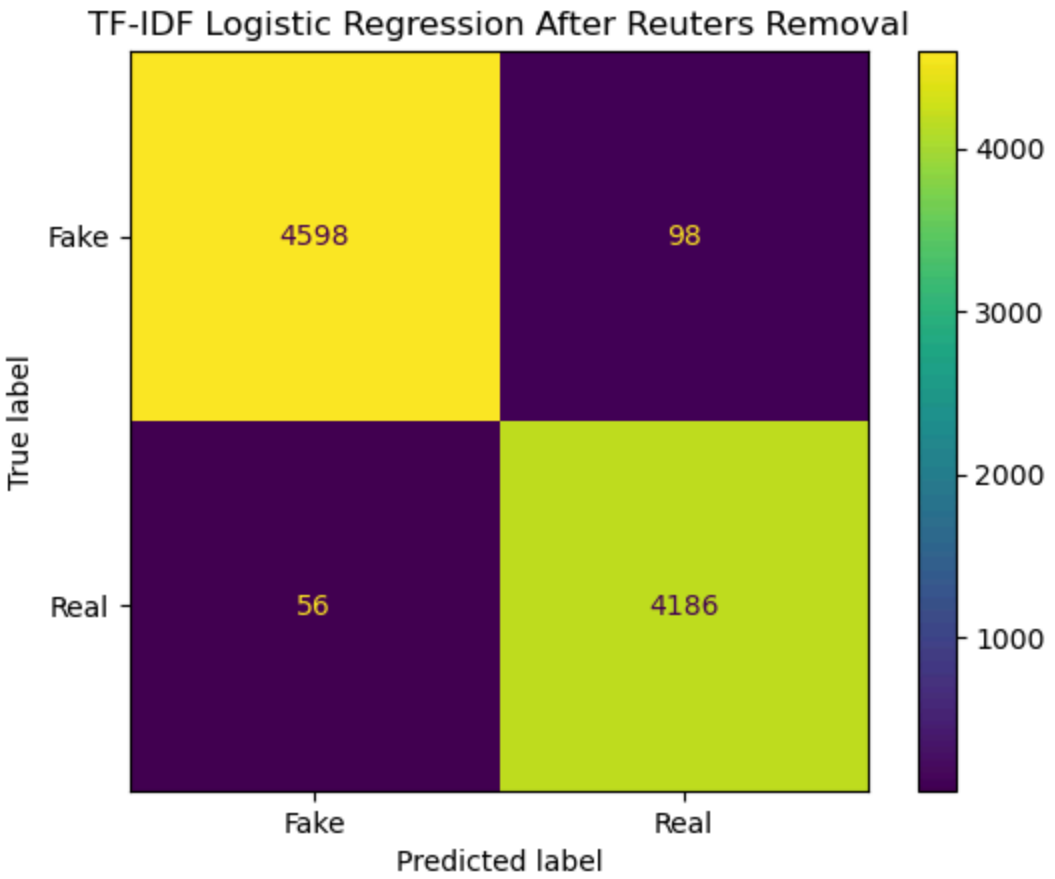
In [34]:

```
# display confusion matrix after Reuters removal

nr_cm = confusion_matrix(y_test_nr, tfidf_lr_predictions_nr)

print(nr_cm)
# what the display has
disp = ConfusionMatrixDisplay(
    confusion_matrix=nr_cm,
    display_labels=["Fake", "Real"]
)
# display the plot
disp.plot()
plt.title("TF-IDF Logistic Regression After Reuters Removal")
plt.show()
```

```
[[4598   98]
 [  56 4186]]
```



After removing the word **Reuters**, the TF-IDF Logistic Regression model achieved an accuracy of **0.9828**, precision of **0.9771**, recall of **0.9868** and F1-score of **0.9819**. This is lower than the original TF-IDF Logistic Regression result, which achieved an accuracy of **0.9886** and F1-score of **0.9880**.

The performance drop shows that the Reuters source marker had some influence on the original model's performance. This supports the concern identified in Section 5, where the word **Reuters** appeared in almost all real news articles but very few fake news articles. Therefore, the original model may have partly used Reuters as a source-related shortcut when classifying articles.

However, the model still performs strongly even after removing Reuters. The F1-score only drops from **0.9880** to **0.9819**, which means the model is not relying entirely on the Reuters marker. It is likely also learning other textual patterns from the article content, such as word choice, writing style and topic-related differences between fake and real news articles.

The confusion matrix after Reuters removal shows that the model correctly classified **4,598 fake news articles** and **4,186 real news articles**. It misclassified **98 fake articles as real** and **56 real articles as fake**. Compared with the original TF-IDF Logistic Regression model, the number of errors increased from **102** to **154**. This confirms that removing Reuters made the classification task more difficult, but the model still remained highly accurate.

Overall, the Reuters removal test strengthens the evaluation because it shows that the very high model scores should be interpreted carefully. The dataset contains source-related patterns that make classification easier, but the model also appears to learn useful language-based features beyond the Reuters marker.

7.5 Comparative Methodology Summary

This section implemented and compared three models for fake news detection. The baseline model used **CountVectorizer with Multinomial Naive Bayes**, providing a simple benchmark based on word-count features. The statistical model used **TF-IDF with Logistic Regression**, providing a stronger word-importance-based representation. The embedding-based model used a **1D CNN**, allowing the classifier to learn dense word representations and local phrase-level patterns from article text.

The results show that all three models performed well, but their performance improved as the text representation and model complexity increased. The baseline model achieved an F1-score of **0.9480**, the TF-IDF Logistic Regression model improved this to **0.9880**, and the 1D CNN achieved the best F1-score of **0.9985**.

These findings suggest that the embedding-based CNN was the strongest model for this dataset. However, the TF-IDF Logistic Regression model also remained highly effective while being simpler, faster and easier to interpret. This is important because a more complex model is not always the best practical choice if a simpler model can achieve very similar performance.

The Reuters removal test also showed that the dataset contains a source-related pattern that influences performance. After removing the word **Reuters**, the TF-IDF Logistic Regression F1-score decreased from **0.9880** to **0.9819**. This confirms that the original model was partly affected by the Reuters source marker. However, the model still performed strongly after removal, suggesting that it also learned useful language-based patterns beyond the Reuters marker.

Overall, the CNN produced the best experimental result, but the very high scores should be interpreted carefully. The results show strong performance on this dataset, but they may not fully represent real-world fake news detection across newer articles or more diverse sources. Therefore, further testing on external datasets would be needed to confirm how well the models generalise beyond this dataset.

8. Programming Style

By using Jupyter notebook in Python, I did this implementation. The notebook is segregated into sections that follow the entier machine learning progress from importing libraries, loading the data, preprocessing texts, training models, evaluating performance and

comparing outputs. This format makes the project easier to pursue and enables every step of the experiment to be checked seperately.

The code uses suitable variable such as **fake_df**, **real_df**, **news_df**, **X_train**, **X_test**, **baseline_model**, **tfidf_lr_model** and **cnn_model**. These names make it clear what each variable represent. Comments are added throughout the notebook to explain crucial steps, adding class labels, removing duplicates, cleaning texts, fitting vectorisers and model trainings.

Reproducibility was acheived by fixing a random state. The value **RANDOM_STATE = 42** was used while shuffling, train-test splitting and model creation whereever its doable. This helps make the output more stable when the notebook is rerun. For the CNN model, TensorFlow and NumPy random seeds were also set to reduce variation between runs, although small differences can still occur in neural network training.

The implementation also avoids data leakage. CountVectorizer and TfidfVectorizer were fitted mainly on the training data for the statistical models, and then it was applied to the testset. This makes sure that vocabulary and the important word informations from the testset are not used during training. Similarly, the Keras Tokenizer for the CNN model was fitted mainly on the training text before being used to alter the test text.

The notebook separates the preprocessing required for statistical models and the embedding-based model. The baseline and TF-IDF models use sparse numerical feature matrices, while the CNN uses tokenised and padded sequences. This distinction is important because different model types require different input formats.

Visualisations are also used to support the analysis. The notebook includes a class distribution chart, confusion matrices for model evaluation and an F1-score comparison chart. These visual outputs help explain the results more clearly than numerical metrics alone.

In total, the programming style is starightforward, flexible and suitable for the project. The code goes through logical sequence, uses comments, avoids testset leakage and stands with reproducible model comparison.

9. Performance Analysis and Comparative Discussion

In conclusion the three models performed well on the fake news detection task and performance improved as the models became more advanced. The baseline CountVectorizer with Multinomial Naive Bayes model got an accuracy of 0.9503 and an F1-score of 0.9480. This strong baseline shows that simple characteristics based on word counts can identify useful distinctions between false and real news articles.

The TF-IDF Logistic Regression model got a better accuracy of 0.9886 and an F1-score of 0.9880. We can understand that TF-IDF was better than simple word counts for representing the article text. TF-IDF decreases the influence of common words and gives more priority to words that are more distinctive within specific documents unlike countVectorizer. This helped the Logistic Regression model segregate fake and real news articles more correctly.

Embedding-based 1D CNN model had the best performance, which got an accuracy of 0.9985 and an F1-score of 0.9985. This shows that the CNN learned very effective word and phrase-level patterns from the article text. The CNN also made lesser errors than the other models, showing that embedding-based approach made the strongest result.

The baseline model was the simplest and quickest, but produced the weakest output. The TF-IDF Logistic Regression model provided a strong balance between performance, simplicity and interpretability. It reduced misclassifications alot compared with the baseline while remaining easier to explain than the CNN. The CNN achieved the highest performance, but it also required more preprocessing steps,tokenisation,sequence padding,neural network training and higher computational cost.

The Reuters removal test gave an important finding. Section 5 showed that Reuters appeared in 99.82% of real news articles but only 1.37% of fake news articles. After removing Reuters, the TF-IDF Logistic Regression model's F1-score decreased from 0.9880 to 0.9819. This shows that the Reuters source marker influenced model performance. However, the model still performed strongly after removal, implying it was not relying entirely on this source pattern and was also learning other language-based features.

Although the CNN achieved the best result, the very high performance should be interpreted carefully. The dataset may contain source-specific, stylistic or topic-related patterns that separate fake and real articles more clearly than in real-world conditions. This means the models may perform extremely well on this dataset but may not generalise equally well to newer articles from different sources. Since this project only uses article text, it also does not include external fact-checking evidence, source credibility information or social media propagation signals.

Overall, the results suggest that embedding-based 1D CNN produced the strongest performance for this experiment. However, the TF-IDF Logistic Regression model remains a highly effective overall and practical alternative because it achieved very high accuracy with lower complexity. Therefore, the choice between the models depends on the intended use case. If maximum predictive performance is the priority, the CNN is the strongest. If interpretability, speed and simplicity are more important, TF-IDF with Logistic Regression may be the better practical choice.

10. Project Summary and Reflections

The fake news detection system was created by using NLP techniques including text classification. The aim was to compare traditional statistical model and deep learning model based on embeddings. For this purpose, the dataset was gathered by combining fake and real news articles and preprocessing the text data.

Three models were developed for the project: baseline, statistical and CNN. The former used Count Vectorizer together with Multinomial Naive Bayes resulting in 0.9503 accuracy. The latter used TF-IDF together with Logistic Regression and showed 0.9886 accuracy. The 1D CNN based on embedding provided 0.9985 accuracy and 0.9985 F1-Score. Thus, it can be stated that all three approaches were efficient but CNN model performed the best.

The project proves the significance of good text representation in the context of text classification task. Word count-based features could be used successfully but TF-IDF gave the more meaningful representation by highlighting unique terms. CNN proved the best performance due to the dense representation of words and phrases. However, TF-IDF Logistic Regression also worked pretty well.

One of the strengths of the work is that the project was not limited to reporting high scores of the models used. For example, the Reuters source-pattern check and Reuters removal test investigated whether there was a source-based feature that would make the dataset easier for separation. The TF-IDF Logistic Regression F1-score decreased from 0.9880 to 0.9819 when Reuters were removed from the training set.

One of the limitations is that the model was trained and evaluated using a single dataset only. This dataset can have source-specific or stylistic features that will help distinguish between fake and true news. Therefore, the high scores of the model may not show how well it can perform on the real-world news published by other sources. Another limitation is the fact that only the text of the article was considered.

Potential improvements in the future would include evaluating the effectiveness of these models using an external dataset of fake news. Additionally, advanced embedding approaches, such as pretrained transformers, could also be investigated. Another possibility is to incorporate metadata with text information in order to develop a more realistic detection model. Overall, the project was successful in achieving its aim of comparing statistical and embedding approaches in NLP models to classify fake news. The findings imply that embedding-based deep learning has the potential to perform best, while statistical models are still robust options.

11. References

[1] Alshuwaier, F.A. (2025) ‘Fake News Detection Using Machine Learning and Deep Learning Algorithms: A Comprehensive Review and Future Perspectives’, Computers, 14(9), Article 394. Available at: https://www.mdpi.com/2073-431X/14/9/394

[2] Roumeliotis, K.I., Tselikas, N.D. and Nasiopoulos, D.K. (2025) 'Fake News Detection and Classification: A Comparative Study of Convolutional Neural Networks, Large Language Models, and Natural Language Processing Models', Future Internet, 17(1), Article 28. Available at: https://www.mdpi.com/1999-5903/17/1/28

[3] Bisaillon, C. (2020) Fake and Real News Dataset. Kaggle. Available at: https://www.kaggle.com/datasets/clmentbisaillon/fake-and-real-news-dataset

[4] Rainio, O., Teuho, J. and Klén, R. (2024) 'Evaluation metrics and statistical tests for machine learning', Scientific Reports, 14, Article 6086. Available at: https://www.nature.com/articles/s41598-024-56706-x

[5] ISOT Research Lab (2023) ISOT Fake News Dataset ReadMe. University of Victoria. Available at: https://onlineacademiccommunity.uvic.ca/isot/wp-content/uploads/sites/7295/2023/02/ISOT_Fake_News_Dataset_ReadMe.pdf

[6] Scikit-learn Developers (2026) Metrics and scoring: quantifying the quality of predictions. Available at: https://scikit-learn.org/stable/modules/model_evaluation.html